Stream Method	What You CAN Do	What You SHOULD NOT Do	Code Example (Valid)	Why / Notes
filter()	✔ boolean checks ✔ null checks ✔ comparisons ✔ method calls returning boolean	✕ modify external variables ✕ I/O operations ✕ object mutation	.filter(e -> e.getAge() > 18 && e.isActive())	Must be **pure predicate**
distinct()	✔ rely on equals() / hashCode() ✔ immutable objects	✕ mutable fields affecting equality	class User { public boolean equals(Object o) { ... } }stream.distinct();	Internally uses HashSet
map()	✔ transform objects ✔ create new objects ✔ call pure functions	✕ mutate input object ✕ change shared state	.map(u -> u.getName().toUpperCase())	One-to-one transformation
flatMap()	✔ return Stream ✔ split data ✔ expand collections	✕ return null ✕ heavy logic inside mapper	.flatMap(s -> Arrays.stream(s.split(\" \")))	One-to-many mapping
sorted()	✔ natural ordering ✔ custom comparator ✔ multi-field compare	✕ comparator with side effects	.sorted(Comparator.comparing(User::getAge))	Stateful + expensive
peek()	✔ logging ✔ debugging ✔ metrics	✕ business logic ✕ mutation	.peek(e -> System.out.println(e))	Debug only
collect()	✔accumulate data	✕ non-thread-safe	.collect(Collectors.toList())	Terminal operation

	✔ grouping ✔ reduction ✔ custom collectors	mutation (parallel)		

```
List<Employee> result =
        employees.stream()                                      // Source
        .filter(e -> e.salary > 50000)                          // FilterStage
        .map(e -> new Employee(e.id, e.name, e.salary))         // MapStage
        .distinct()                                             // DistinctStage
        .sorted(Comparator.comparingInt(e -> e.salary))         // SortedStage
        .peek(e -> System.out.println("After sort: " + e))      // PeekStage
        .collect(Collectors.toList());                          // CollectOperation
    System.out.println("Final Result: " + result);
```